Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

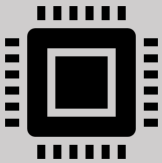
División de Ciencias de la ingeniería

Ingeniería

Organización de Lenguajes y Compiladores 2 Sección “A”

Ing. Mario Moises Ramirez Tobar y Auxiliar Yennifer de León

Segundo Semestre 2026



Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Manual de usuario Proyecto #2



Descripción:

En el siguiente manual se describe paso a paso cómo realizar la instalación de la aplicación, esta consiste en un validador y traductor de un lenguaje de programación en latin y permite la visualización de los procesos y diferentes fases que realiza un compilador para llegar a procesar un código de entrada en cierto lenguaje a un código de bajo nivel de una representación de código intermedio y genera código ensamblador. También esta aplicación está realizada con una interfaz gráfica intuitiva para el usuario donde el puede acomodar las vistas como le sean útiles y en ningún momento se le obliga a tener cierta pestaña abierta. Espero sea de su agrado.

“Bienvenido usuario”

Como primer punto se requieren ciertos recursos básicos para poder ejecutar el programa. Debido a las tecnologías usadas como aplicación de escritorio, esta cuenta con procesos optimizados lo cual no conlleva una demanda bastante grande en requerimientos para ejecutar la aplicación.

Recomendaciones:

Debido a que la aplicación está diseñada para escritorio se debe tomar en cuenta que no está disponible para dispositivos móviles a no ser que el usuario desee correr allí. Los requerimientos básicos son los siguientes:

- Teclado y mouse.
- El sistema operativo queda a decisión del usuario.
- **Una pantalla lo suficientemente grande para poder visualizar con comodidad las diferentes representaciones gráficas que contiene la aplicación.**

Pasos iniciales:

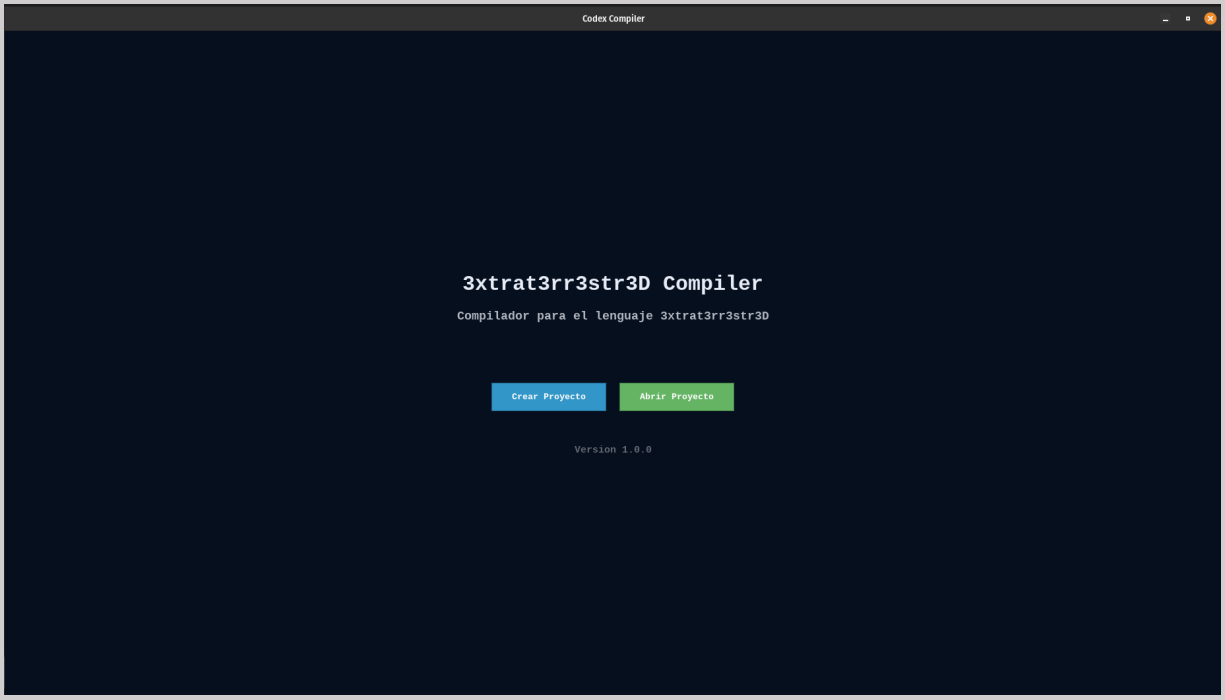
1. Encender el dispositivo
2. Descargar el archivo .zip o clonar el repositorio
3. Extraer el archivo comprimido o navegar hasta el directorio destino
4. En la raíz del programa o en la carpeta destino abierta.
5. Ejecutar el comando **mvn clean package**
6. Navegar hasta la carpeta que se va a crear llamada build y entrar a la carpeta **target**
7. Abrir la terminal y ejecutar el siguiente comando

java -jar proyecto-no1-compi2-1.0.0.jar

Si en dado caso la aplicación le pide permisos para poder acceder a sus archivos **debe darle aceptar ya que esta tiene la opción de poder cargar proyecto siempre y cuando tengan la extensión .y, .z y .pig para poder exportar los proyectos y manejar los archivos**

MENÚ PRINCIPAL

Apenas ejecutar el comando en la terminal del dispositivo saltará la interfaz de inicio lo que permite generar una primera vista de cómo está hecha la interfaz. El tema está basado completamente en un tema oscuro para evitar ciertos problemas de visualización con un tema claro:



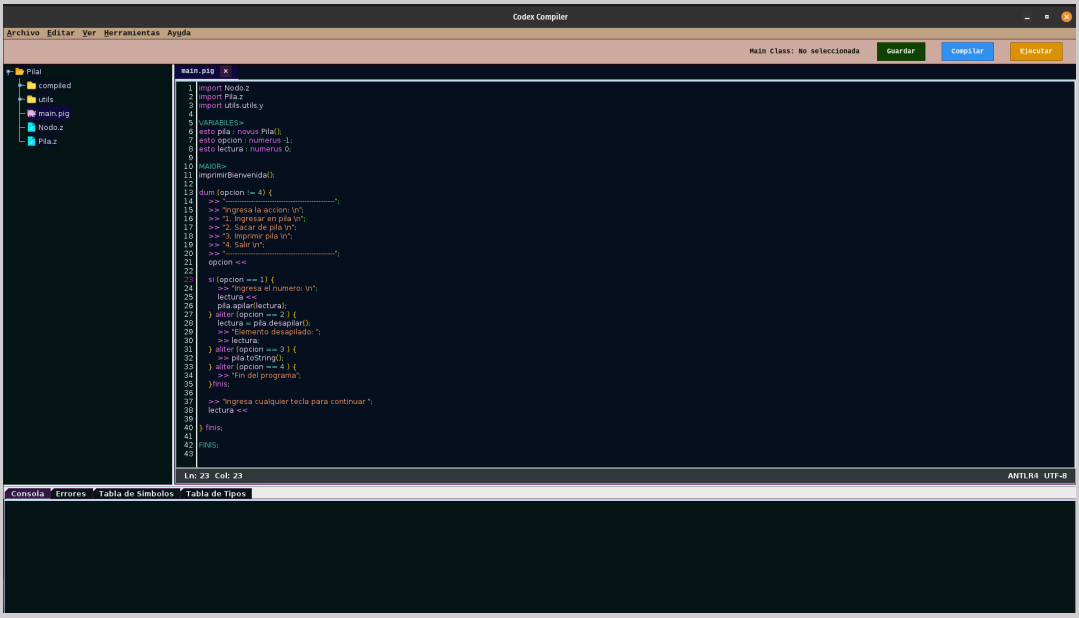
Nota:

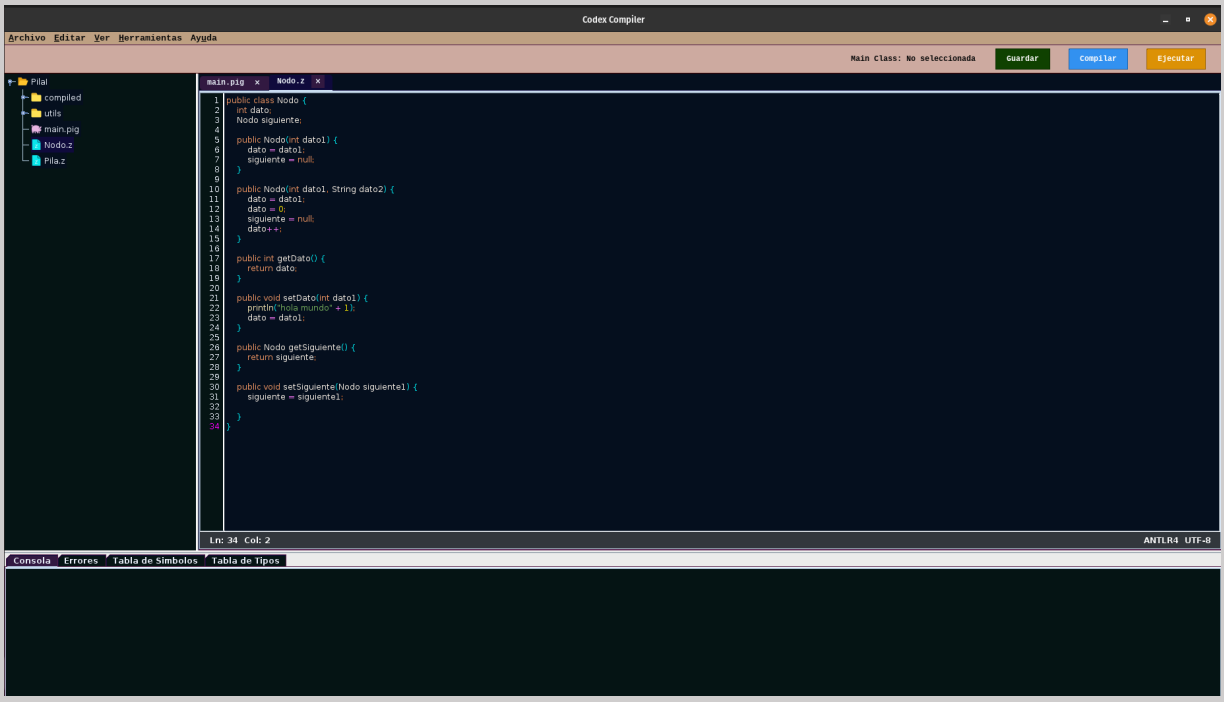
La interfaz de usuario cuenta con una tipografía específica llamada “Liberation mono” que da esa estética retro pero a la vez elegante de un lenguaje de programación. También no está demás tomar en cuenta que:

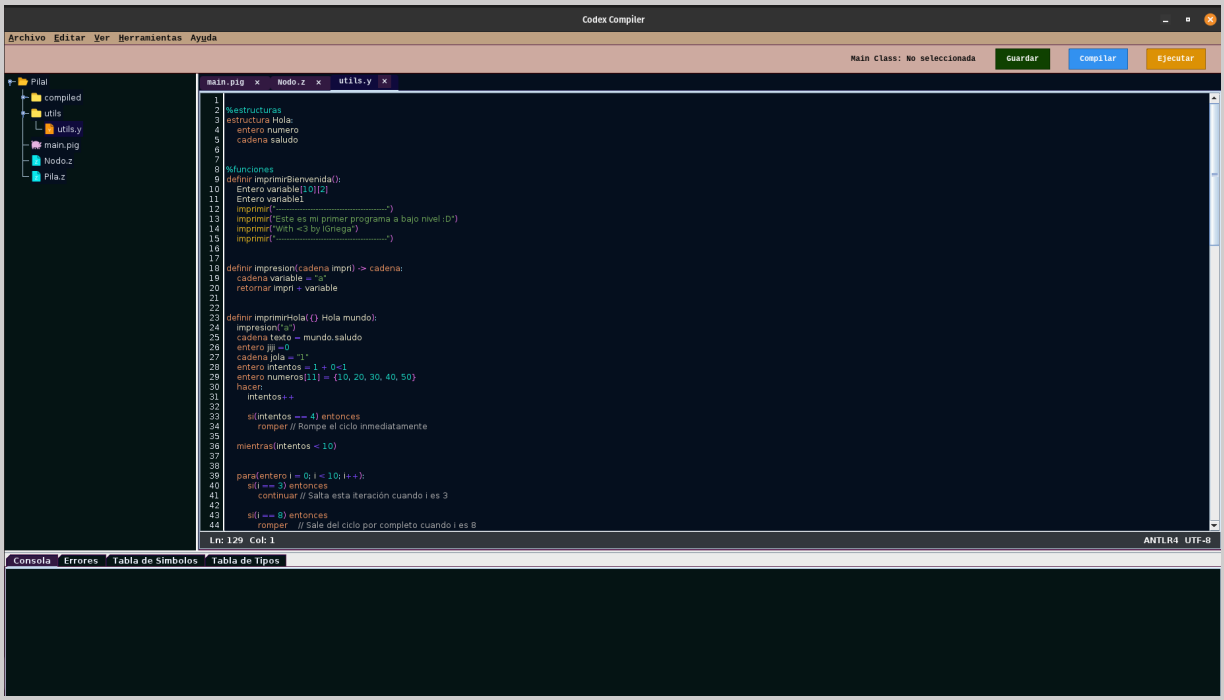
“Todas las vistas son redimensionables y ajustables a gusto del usuario”

COLOREADO DINAMICO DEL CÓDIGO

Algo que se podrá notar apenas empezar a escribir el código en la aplicación. Es que cuenta con coloreado dinámico para poder tener una mejor vista de las palabras reservadas y valores a la hora de programar. brindando una interfaz amigable.





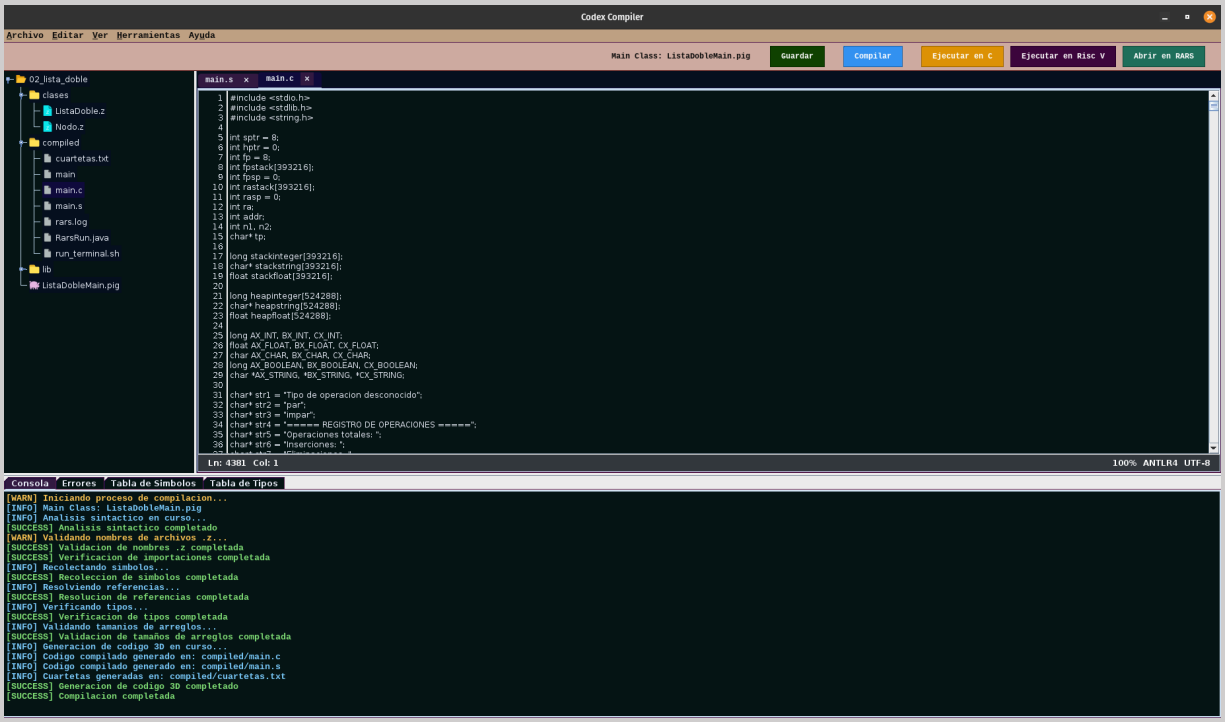

Codex Compiler
Archivo Editar Ver Herramientas Ayuda
Main Class: No seleccionada
Guardar
Compilar
Ejecutar
Pilal
compiled
utils
utils.y
main.pig
Nodo.z
Pila.z
main.pig
Nodo.z
utils.y
%estructuras
estructura Hola:
entero numero
cadena saludo
%funciones
definir imprimirBienvenida():
Entero variable[10][2]
Entero variable1
imprimir("---")
imprimir("Este es mi primer programa a bajo nivel :D")
imprimir("With <3 by IGriega")
imprimir("---")
definir impresion(cadena impri) -> cadena:
cadena variable = "a"
retornar impri + variable
definir imprimirHola(){} Hola mundo):
impresion("a")
cadena texto = mundo.saludo
entero jiji =0
cadena jola = "1"
entero intentos = 1 + 0<1
entero numeros[11] = {10, 20, 30, 40, 50}
hacer:
intentos++
si(intentos == 4) entonces
romper // Rompe el ciclo inmediatamente
mientras(intentos < 10)
para(entero i = 0; i < 10; i++):
si(i == 3) entonces
continuar // Salta esta iteración cuando i es 3
si(i == 8) entonces
romper // Sale del ciclo por completo cuando i es 8
Ln: 129 Col: 1
ANTLR4 UTF-8
Consola
Errores
Tabla de Símbolos
Tabla de Tipos

GENERACIÓN DE ARCHIVOS COMPILADOS

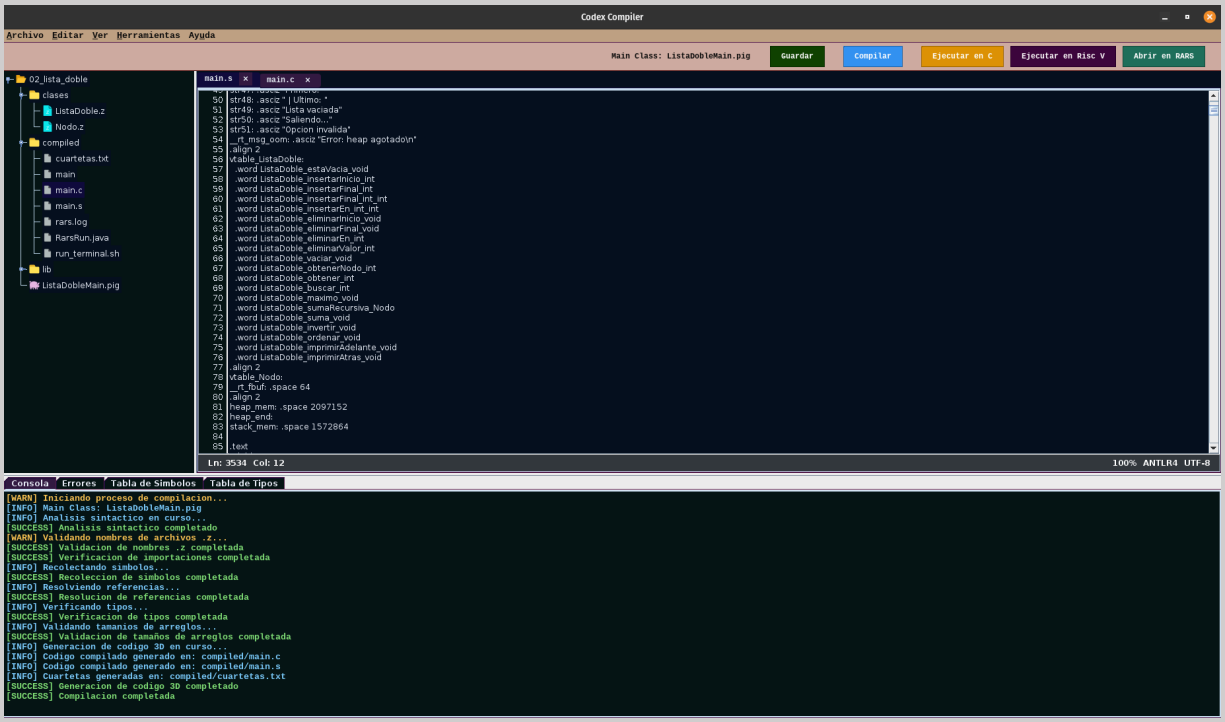
La aplicación tras generar todo el proceso de compilación este genera un código de salida donde se genera el ejecutable compilado tanto a ensamblador RISC V y a código de tres direcciones en C.

Esta es una forma en la que el usuario puede visualizar directamente como es que un compilador desde un código de alto nivel se puede llegar a representar a codigo de muy bajo nivel. Ofreciendo herramientas de compilación en terminal o en la aplicación de RARS.

Código de tres direcciones en C:

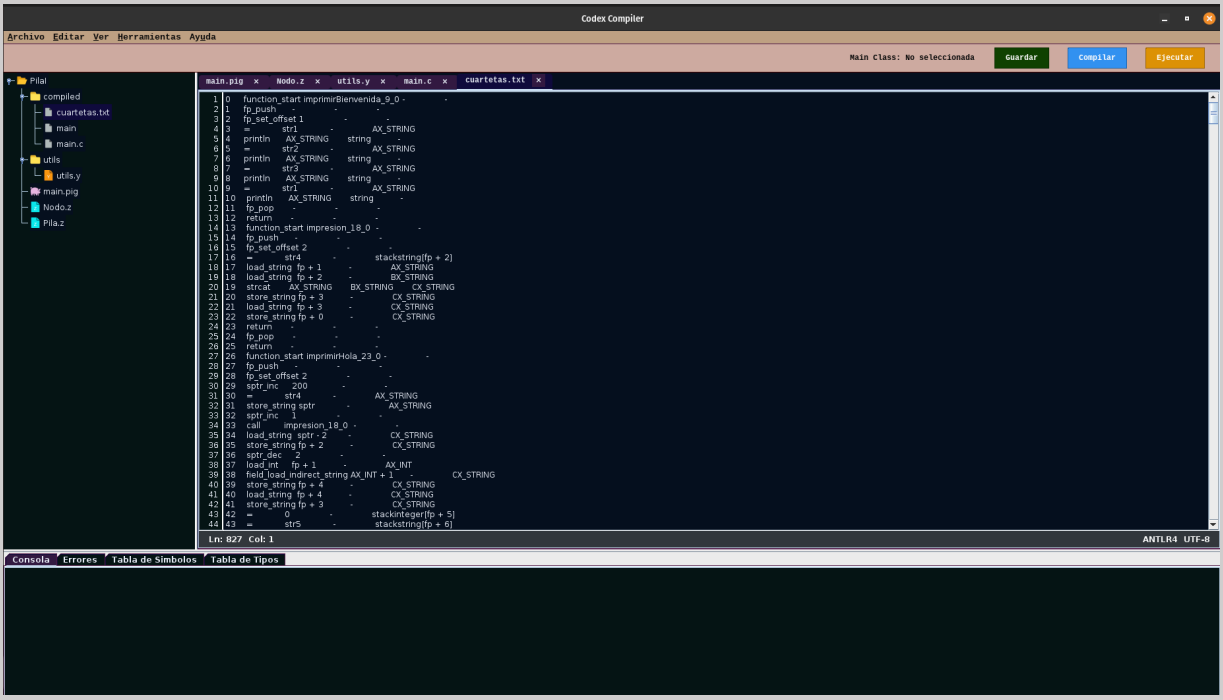


Código en RISC V:



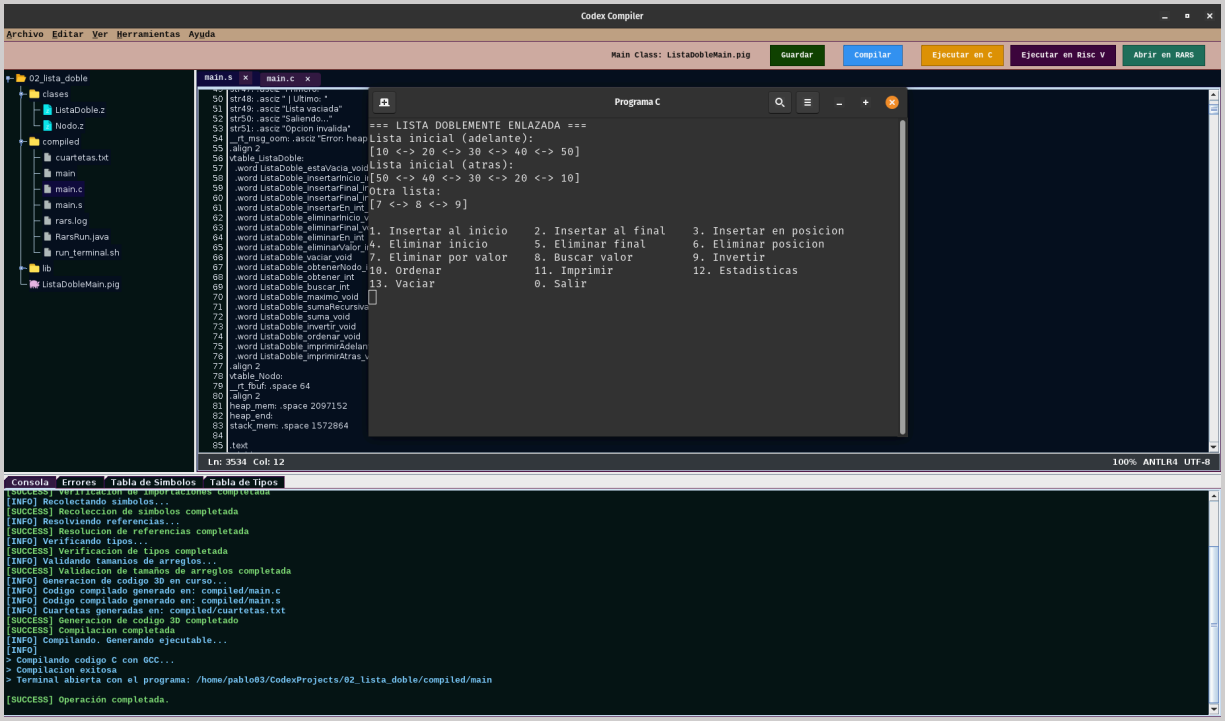
GENERACIÓN DE CUARTETAS

Cada vez que se compila un nuevo código. El programa genera una representación de cuartetas donde también al usuario se le permite ver la representación de código intermedio previo que hace el compilador antes de llegar a código de bajo nivel.



EJECUCIÓN DE CÓDIGO C

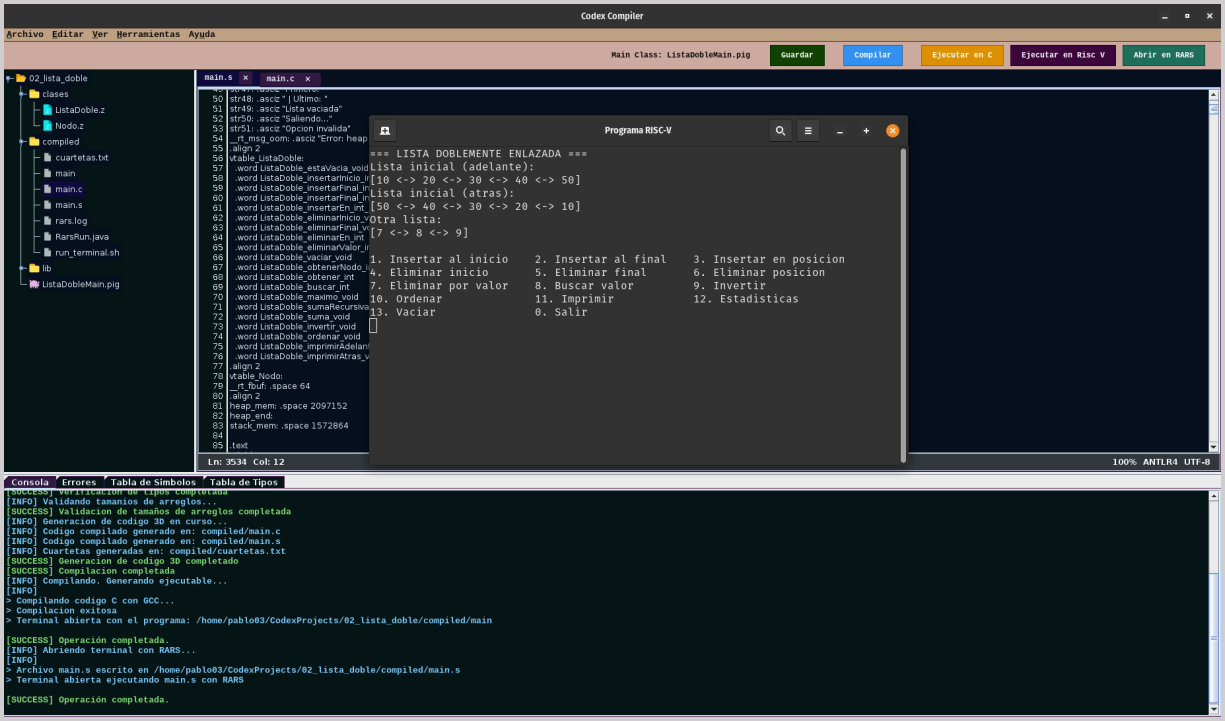
Se puede ejecutar el código directamente desde la terminal. La aplicación dispone de funcionalidades integradas que compilan automáticamente el código de C desde GCC y lo corren en una consola.



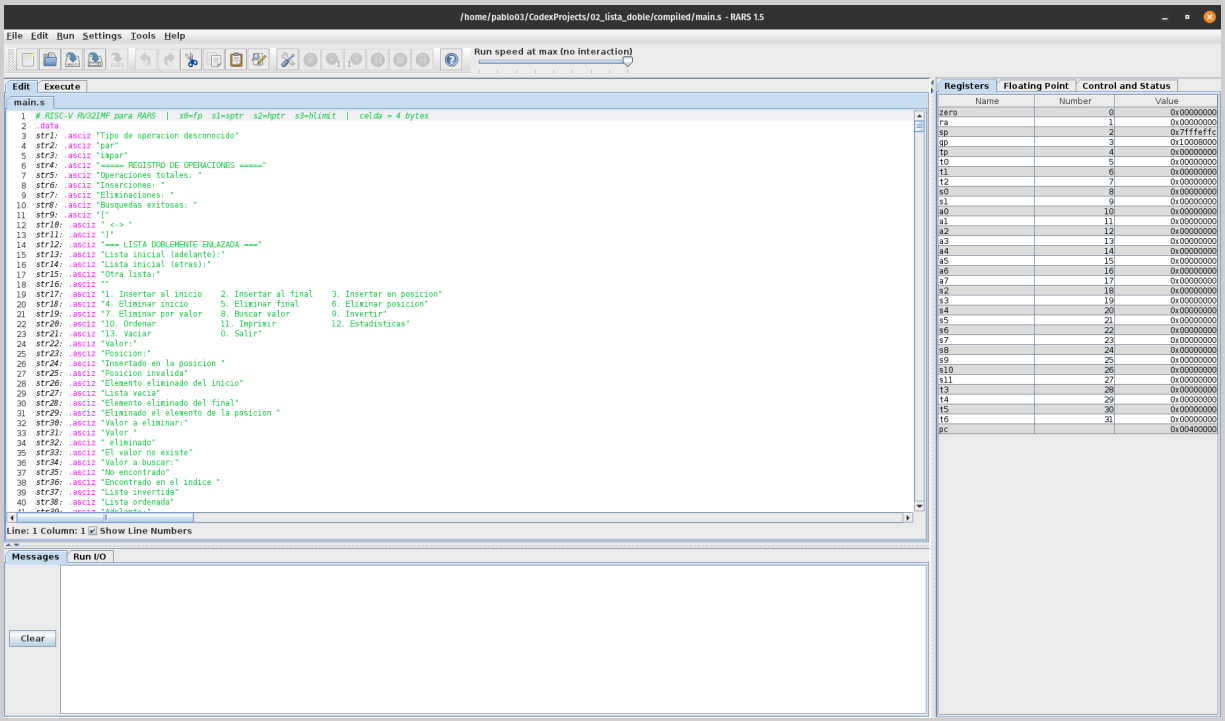
EJECUCIÓN DE CÓDIGO RISC V

Se puede ejecutar el código directamente desde la terminal o también se puede utilizar la herramienta de RARS. Que es un entorno completo para poder ejecutar y visualizar la forma en la que se generan los registros de ensamblador. De esta forma de generar un ecosistema interactivo para que el usuario pueda apreciar cómo es que trabaja la computadora a bajo nivel

Ejecución en consola:



Ejecución en RARS:



Visualización de direcciones de memoria:

